LLM 显存占用分析

训练与推理过程中的内存需求与优化

Bilibili 以往的月

Homepage

2026 年 3 月 19 日

目录

1. Motivation
2. Background
3. Training Memory Analysis
4. Inference Memory Analysis
5. Optimization
6. Real Systems
7. Conclusion
8. References

目录

1. Motivation
2. Background
3. Training Memory Analysis
4. Inference Memory Analysis
5. Optimization
6. Real Systems
7. Conclusion
8. References

Why Memory Matters

A simple problem

假如我有一张 80GB 的显卡，我想训练/推理一个 4B 的模型，我应该设置多大的 batch size 和 sequence length?

在这个 tutorial 中，我们将基于这个问题来进行思考和分析。我们考虑更一般的问题形式：

Motivation

在训练和推理时 LLM 所需要的内存是多少？如何进行优化内存占用？

我们将先介绍训练/推理阶段的内存计算，再针对可优化部分进行分析并介绍相应优化算法。

目录

1. Motivation
2. Background
3. Training Memory Analysis
4. Inference Memory Analysis
5. Optimization
6. Real Systems
7. Conclusion
8. References

Transformer Architecture

以 Qwen3[YLY+25] 为例，现代 LLM 的架构如右图所示。其中具体的模块不同的模型可能有改动。

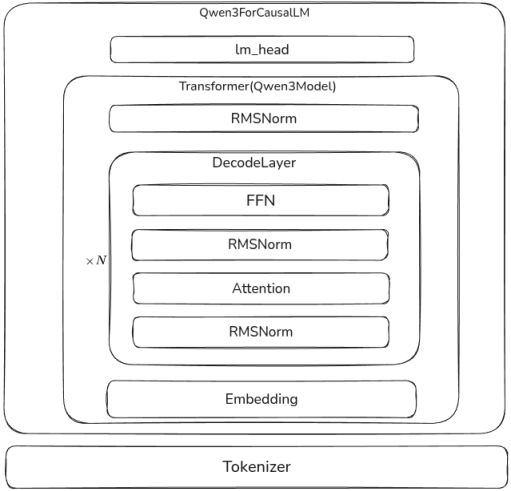


图: Architecture of Qwen3

Notation

与参数量、FLOPs 计算所用记号一致；参数量 P 的推导见 LLM parameter analysis。

变量	含义
P	number of parameters
L	layers
V	vocabulary size
d	hidden size
d_{ff}	FFN hidden size
s	sequence length
b	batch size
h	number of attention heads
d_h	attention head dimension

Assumption

1. 若无特别说明，使用 **BF16/FP16**，每个参数 2 byte。
2. 不使用 dropout（与现代大模型设定一致）。
3. Attention 基于原始 multi-head attention.
4. FFN 基于 SwiGLU.

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

Training Memory Componenets

训练部分的内存占用由四部分组成，如下所示

$$
\begin{aligned}
\text{training_memory} &= \text{weight} && (1)\\
&+ \text{activation} && (2)\\
&+ \text{optimizer} && (3)\\
&+ \text{gradient} && (4)
\end{aligned}
$$

Weights: $\boxed{2P}$.

Gradients（与权重同精度）: $\boxed{2P}$.

Optimizer states

AdamW 优化器：

- **一阶动量**：$2P$,
- **二阶动量**：$2P$,
- **合计**：$2 \times 2P = \boxed{4P}$.

Algorithm 1: AdamW[LH19]

Input: $\alpha,\ \lambda,\ \beta_1, \beta_2 \in [0, 1),\ \epsilon,\ \theta_0$

Output: θ_t

$m_0 \leftarrow 0,\ v_0 \leftarrow 0,\ t \leftarrow 0;$

while *Not converge* **do**

$t \leftarrow t + 1;$

$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1});$

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t;$

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2;$

$\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t};$

$\hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t};$

$\theta_t \leftarrow \theta_{t-1} - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right);$

Activation

Motivation

激活值是前向传播过程中计算得到的中间结果，用于在反向传播时计算梯度。

我们仅针对 linear layer 进行推导

forward
$$\mathbf{z}_\ell = W_\ell \mathbf{a}_{\ell-1} + b_\ell, \quad \mathbf{a}_\ell = \phi(\mathbf{z}_\ell)$$

backward
$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_\ell} \cdot \frac{\partial \mathbf{z}_\ell}{\partial W_\ell} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_\ell} \boxed{\mathbf{a}_{\ell-1}}$$

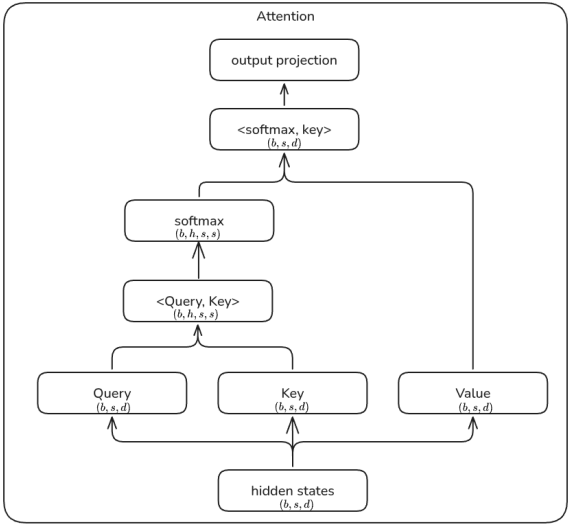
可以看到，计算第 ℓ 层关于 W_ℓ 的梯度时需要其输入 $\mathbf{a}_{\ell-1}$，因此训练时需保存每个模块对应的输入，也就是激活值 (activation).

Activation-Attention

按计算图（无优化）可得需保存的激活：

- Q/K/V 投影：共享输入 $\to 2bsd$
- $Q^\top K$：Q, K 均需保存 $\to 2 \times 2bsd = 4bsd$
- softmax 输入：$2bhs^2$
- weighted sum 输入：$2bhs^2 + 2bsd$
- output projection 输入：$2bsd$

Attention 合计：$\boxed{10sbd + 4bhs^2}$.

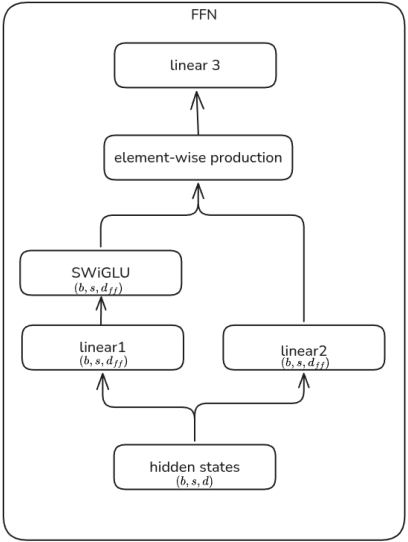


Activation-FFN & LayerNorm

FFN（SwiGLU，assume $d_{\mathrm{ff}} = 4\,d$）:

- 第一层输入 $2\,sbd$
- SwiGLU 输入 $2 \times d_{ff} \times d = 8\,sbd$
- 第二层输入 $2 \times d_{ff} \times d = 8\,sbd$
- 合计：$\boxed{18sbd}$

LayerNorm：保存输入 $\rightarrow$ $\boxed{2bsd}$.

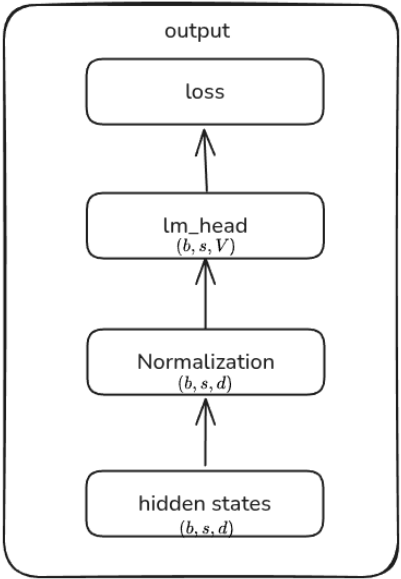


Activation-Output

Output 包含以下组成部分：

- **FinalNorm** 输入：$2bsd$
- **lm_head** 输入：$2bsd$
- **Loss** 输入：$2bsV$

合计：$\boxed{4bsd + 2bsV}$。



Activation-Total

我们将上面的结果汇总在一起，得到[1]

$$
\begin{aligned}
\text{activation} &= L \cdot \text{transformer_block} + \text{output} && (5)\\
&= L \cdot (\text{Pre_Norm} + \textit{Attention} + \text{Post_Norm} + \text{FFN}) + \text{output} && (6)\\
&= \boxed{bs(32dL + 4hsL + 4d + 2V)} && (7)\\
&\approx bsL(32d + 4hs) && (8)\\
&\approx 4bs^2hL && (9)
\end{aligned}
$$

我们可以看到，未优化的情况下，$\text{activation} \propto bs^2$. 这里 s^2 主要由 attention 部分产生，后续 flash attenion 就针对这一点进行了优化。

[1] Qwen3 中，$2V/32dL = 6\%$, $32dL/4hsL = 2.5\%$.

Total Training Memory

我们将上面的结果进行汇总得到

$$
\begin{aligned}
\text{training_memory} &= \text{weight} + \text{activation} + \text{optimizer} + \text{gradient} && (10)\\
&= 2P + bs(32dL + 4hsL + 4d + 2V) + 4P + 2P && (11)\\
&= 8P + bs(32dL + 4hsL + 4d + 2V) && (\text{exact})\\
&\approx 8P + 4bs^2hL && (12)
\end{aligned}
$$

可以看到，训练阶段的内存占用分为固定部分 ($8P$) 和动态部分 ($4bs^2hL$)，动态部分主要是 attention 的缓存。

Analysis

我们以 Qwen3 模型为例，得到如下热力图，batchh size 越小，说明 activation 占用越高

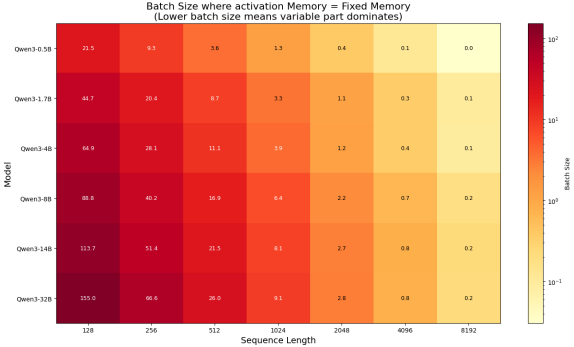


图: Comparison of activation memory with fixed memory

Experiments

我们分别针对 80GB 的显卡计算 Qwen3 系列模型的最高配置：

Model	P	L	h	s	predicted b	actual b
Qwen3-0.6B	0.6	28	16	512	68	34
Qwen3-1.7B	1.7	28	16	512	42	28
Qwen3-4B	4	36	32	512	16	12
Qwen3-8B	8.1	36	32	512	4	2

其中 predicted b 基于前面的准确公式计算得到；actual b 通过实验验证得到。（注意我们这里的 prediction 没有考虑任何优化手段与其他内存开销，因此与实际值有出入）

Case Study

我们分别使用 Qwen3-4B 和 Qwen3-8B 来进行实验 ($b = 1$, $s = 512$), 结果如下[2]

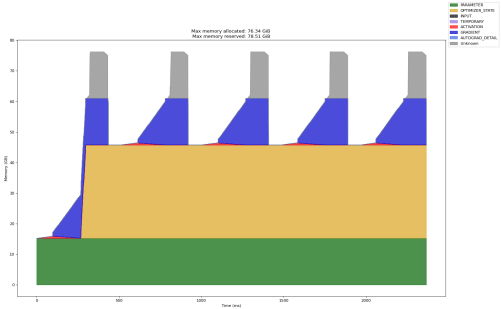


图: Memory profile of Qwen3-8B

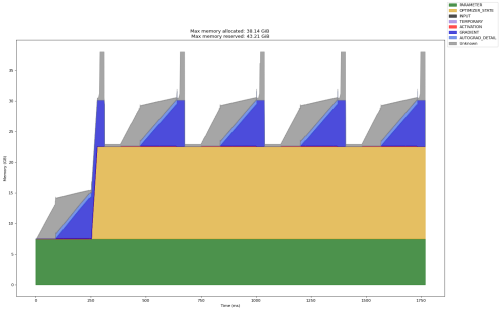


图: Memory profile of Qwen3-4B

[2]参考PyTorch 显存可视化与 Snapshot 数据分析

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

Inference Components

Inference 阶段内存占用主要与模型参数，KV cache 两部分相关。

$$\text{Inference_Memory} = \text{weight} \tag{13}$$
$$+ \text{activation} \tag{14}$$
$$+ \text{KV cache} \tag{15}$$

- Weights：$\boxed{2P}$
- Activations（经验[3]，batch size=1）：$\approx 0.4P$
- KV cache：与序列长度相关，见后续分析。

[3] 见 https://blog.eleuther.ai/transformer-math/

KV Cache Mechanism

Motivation

LLM 推理中为避免重复计算历史 token 的 key/value 而使用的**空间换时间**的缓存机制。

自回归时逐 token 生成，每步 attention 形式为（$\mathbf{q}_t$ 当前 query，$\mathbf{k}_{:,t}/\mathbf{v}_{:,t}$ 历史 K/V）：

$$\mathbf{q}_t = W_Q\mathbf{x}_t, \quad \mathbf{k}_{:,t} = W_K[\mathbf{x}_1, \ldots, \mathbf{x}_t], \quad \mathbf{v}_{:,t} = W_V[\mathbf{x}_1, \ldots, \mathbf{x}_t]$$

处理下一 token $\mathbf{x}_{t+1}$ 时只需在已有结果后追加当前步：

$$\mathbf{k}_{:,t+1} = W_K[\mathbf{x}_1, \ldots, \mathbf{x}_{t+1}] = [\mathbf{k}_{:,t},\ W_K\mathbf{x}_{t+1}], \quad \mathbf{v}_{:,t+1} = W_V[\mathbf{x}_1, \ldots, \mathbf{x}_{t+1}] = [\mathbf{v}_{:,t},\ W_V\mathbf{x}_{t+1}]$$

缓存前，每生成一个 token 都重新计算 $\rightarrow$ 总计算量 $\sum_{t=1}^{s} \mathcal{O}(t) = \mathcal{O}(s^2)$；

缓存后，每步只算当前 token $W_K\mathbf{x}_{t+1}$、$W_V\mathbf{x}_{t+1}$ $\rightarrow$ 计算量 $\mathcal{O}(s)$、空间占用 $\mathcal{O}(s)$ ——**以空间换时间**。

KV Cache Memory

对于 multi-head attention ，KV cache 的显存占用为：

$$\text{Memory(KV cache)} = s \times 2 \times 2 \times L \times h \times d_h = \boxed{4sLhd_h} \tag{16}$$

因子含义： s 序列长，2 为 K+V, L 层、h 头、d_h 头维度。

Remark

- KV 占用与**模型配置**（L, h, d_h）和**序列长度** s 都有关，token 越多占用越高。
- 实际中因 *page granularity*、*padding*、*fragmentation* 往往略高于理论值。
- 长输出时 KV 占比会超过权重，成为推理瓶颈 → 见后续「KV Cache 优化」。

Inference Memory

综合前面分析，我们可以得到推理阶段的总内存为：

$$\boxed{\text{Inference_Memory} \approx 2.4P + 4sLhd_h}$$

可以看到，推理阶段也由固定部分（参数量，activation）以及动态部分（KV cache）组成。

Experiments

我们仍然使用 Qwen3 系列来进行实验（未考虑任何优化，仅考虑 KV cache 和 prefilling），结果如下：

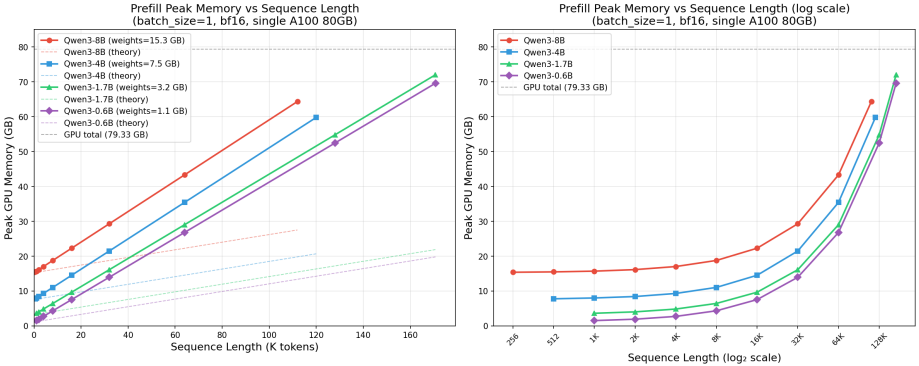


图: Memory profile for Qwen3 series

注意这里的 peak memory 实际上并不能和 KV cache 的显存占用完全对应，因为 peak memory 还包括了 attention 的临时显存等。

Dynamic v.s. Static

接下来我们看一下模型权重与 KV cache 的显存占用情况。由于 Qwen3 的 KV cache 计算为 $4sLh_{kv}d_h$，而不同模型只有 L 不一样，因此对于更大的模型，KV cache 显存占用超过模型权重的上下文长度更高。

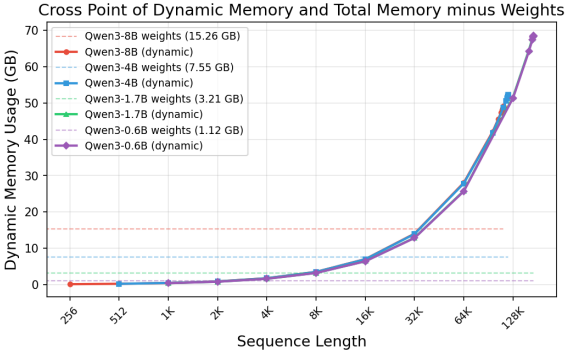


图: Dynamic vs. Static memory profile for Qwen3 series

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

Overview

阶段	核心方法	典型技术
Training	显存与效率提升	Activation Checkpointing Mixed Precision Training Flash Attention ZeRO Pipeline/Model/Data Parallelism
Inference	长序列与速度优化	KV Cache Optimization Paged/Radix Attention Faster Attention Quantization

Mixed Precision Training

Motivation

计算量大的部分用低精度，计算量小的部分用高精度。低精度参与运算，高精度避免 Overflow/Underflow.

下图是 DeepSeek-V3[DALF+25] 使用的混合精度训练框架

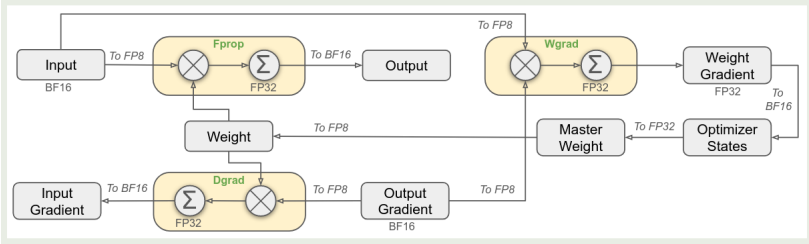


图: Mixed precision training of DeepSeek-V3

Mixed Precision Training-Analysis

Precision	**BF16**	**FP32**	**BF16**
AMP	No	Yes	Yes
Weights	BF16 (2)	FP32 (4)	BF16 (2)
Master weigths	-	-	FP32 (4)
Gradients	BF16 (2)	FP32 (4)	BF16 (2)
Adam m	BF16 (2)	FP32 (4)	FP32 (4)
Adam v	BF16 (2)	FP32 (4)	FP32 (4)
Static total (bytes/param)	8	16	16

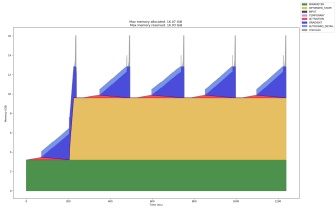
表: Comparison of memory usage per parameter across different training configurations.

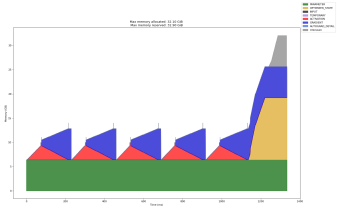
Remark

1. BF16 (w/ AMP) 与 FP32 (w/ AMP) 的静态显存占用相同，但 BF16 (w/ AMP) 的动态显存占用更低。
2. 主流框架基本都使用了 BF16/FP8 (w/ AMP) 的训练方式。

Mixed Precision Training-Experiments

Experiments on Qwen3-1.7B, batch size $= 1$, sequence length $= 512$.





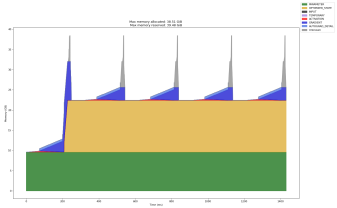


图: (left) BF16 (w/o AMP), $\sim 8P$ bytes, (center) FP32 (w/ AMP), $\sim 16P$ bytes, (right) BF16 (w/ AMP), $\sim 16P$ bytes

ZeRO[RRRH20a]

Motivation

将 optimizer states / gradients / weights 按不同 GPU 切片存储，需要参与计算时再 all-gather 整合成完整参数。这样每张卡只需维护自己负责的一部分，大幅降低单卡显存需求。

ZeRO 的工作原理如下图所示

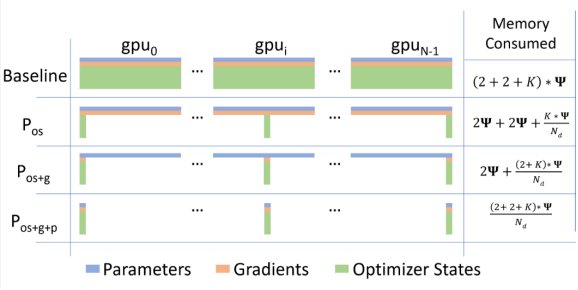


图: ZeRO DP framework

ZeRO Stages

- **ZeRO1**: shard optimizer states

$$\text{Training_Memory} = \text{weight} + \text{activation} + \frac{\text{optimizer}}{\#\text{GPUs}} + \text{gradient} \tag{ZeRO1}$$

- **ZeRO2**: shard gradients + gradients

$$\text{Training_Memory} = \text{weight} + \text{activation} + \frac{\text{optimizer} + \text{gradient}}{\#\text{GPUs}} \tag{ZeRO2}$$

- **ZeRO3**: shard all

$$\text{Training_Memory} = \text{activation} + \frac{\text{weight} + \text{optimizer} + \text{gradient}}{\#\text{GPUs}} \tag{ZeRO3}$$

Remark

ZeRO3 可极大降低单卡显存上限，但通信量也会提高。

Model Parallelism[SPP+20]

Motivation

将模型切分到不同的 GPU 上，计算时，先 dispatch, 再执行计算，最后通过 all-gather 等操作得到最终结果。切分方式包括 PP (Pipeline Parallelism), TP (Tensor Parallelism), EP (Expert Parallelism) 等。

$$\text{Training_Memory} = \frac{\text{Memory(weight)}}{\text{PP degree} \times \text{TP degree}} \tag{17}$$

结合 ZeRO1 与 Model Parallelism 时（activation 中与 TP 相关的部分按 TP degree 缩减）:

$$\text{Memory}_{\text{train}} \approx \frac{\text{weight}}{\text{PP} \times \text{TP}} + \frac{\text{activation}}{\text{TP}} + \frac{\text{optimizer}}{\#\text{GPUs}} + \frac{\text{gradient}}{\text{PP}}$$

Activation Checkpointing[KCL+22]

Motivation

在反向传播时，重新计算所需的输入，来达到以时间换空间的目的。

	No ckpt	Selective ckpt	Full ckpt
memory	很高	中等	很低 $\sim 2bsd$
extra compute	无	中等	很高 $\sim 2Pbs$

Remark

实践中常结合 model parallelism 与 selective checkpointing 来实现 trade off.

Flash Attention[DFE+22]

Motivation

通过将 Attention 的计算进行分块，来提高内存访问效率以及降低反向传播时所需要的 activation 大小。

Flash Attention 通过 tiling 与 online-softmax 降低该部分显存并提升效率[4]。这样 attention 部分的显存就由 $\text{activation} \propto bs^2$ 降低到了 $\text{activation} \propto bs$.

Theorem

flash attention 输出 $O = \mathrm{softmax}(QK^T)V$ (correctness).
其时间复杂度为 $\mathcal{O}(s^2 d)$, 空间复杂度为 $\mathcal{O}(s)$ (memory savings).

[4]详见 notes on Flash Attention

KV Cache Optimization[LLT+25]

$$\text{Memory(KV cache)} = s \times 2 \times 2 \times L \times h \times d_h$$

1. s: KV cache compression, eviction, selection
2. 2: KV cache quantization
3. 2: key-value sharing, MLA[DALF+24]
4. $h \times d_h$: attention, MQA[Sha19], GQA[ALTdJ+23], MLA.

Weight Quantization

Motivation

使用低精度来表示高精度数值的方法，来减少内存占用/提高计算效率。

量化时机	代表性工作
训练后量化 (PTQ)	GPTQ[FAHA23] AWQ[LTT+24] SmoothQuant[XLS+24] GGUF[Ger23]
量化感知训练 (QAT)	LLM-QAT[LOZ+23] PEQA[KLK+23]

表: 大模型量化方法总结

Activation Offloading

Motivation

将一部分参数/优化器状态/激活值等存储到 CPU 上，需要的时候再加载到 GPU 上。

Offloading 场景	代表性工作
训练阶段 Offloading	ZeRO-Offload[RRA+21] / ZeRO-Infinity FSDP[ZGV+23] CPU Offload
推理阶段 Offloading	FlexGen[SZY+23] vLLM[KLZ+23] KV Cache Offload
MoE Offloading	KTransformers[CXZ+25] DeepSpeed-MoE[RLY+22]

表: offloading methods

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

Training

Framework	Memory Optimizations
Megatron-LM[SPP+20]	TP, SP, Activation Checkpointing
DeepSpeed[RRRH20b]	ZeRO-1/2/3, CPU/NVMe Offload, Activation Checkpointing
FSDP[ZGV+23]	Full parameter sharding, Gradient sharding, CPU Offload
Colossal-AI[LLB+23]	ZeRO, TP, PP, Activation Checkpointing

Inference

Framework	Key Techniques	Memory Optimizations
vLLM[KLZ+23]	Paged Attention	KV cache paging, Continuous batching
SGLang[ZYX+24]	Radix Attention	KV cache reuse, Efficient scheduling
TensorRT-LLM[Cor23]	Kernel fusion	Weight quantization, KV cache optimization

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

Takeaway

Components	Training	Inference	Optimization
weights	$2P$	$2P$	quantization
optimizer states	$4P$	0	ZeRO, offloading
gradients	$2P$	0	ZeRO
activations	$\sim 4Lhs^2b$	$\sim 0.4P$	ckpt, offloading, flash attention
KV cache	0	$4sLhd_h$	kv optimization, attention
TOTAL	$8P + 4Lhs^2b$	$2.4P + 4sLhd_h$	

- **训练**：瓶颈主要在激活值（随 batch size / 序列长度线性增长）。
- **推理**：瓶颈主要在 KV Cache（随序列长度增长）。

Conclusion & Future Directions

Conclusion

We discussed how to analyze the memory occupied by an LLM and how to optimize it.

Future Directions

1. More efficient architecture (attention, MoE).
2. Scalable training/inference framework
3. Software-hardware co-design algorithms.

目录

1 Motivation

2 Background

3 Training Memory Analysis

4 Inference Memory Analysis

5 Optimization

6 Real Systems

7 Conclusion

8 References

References I

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai, *Gqa: Training generalized multi-query transformer models from multi-head checkpoints*, 2023.
- NVIDIA Corporation, *TensorRT-LLM: A TensorRT toolset for optimizing LLM inference*, https://github.com/NVIDIA/TensorRT-LLM, 2023.
- Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang, *Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models*, Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, 2025.

References II

DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen,

References III

Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yuduan Wang, Yuheng Zou, Yukun Zha, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie, *Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model*, 2024.

References IV

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan,

References V

T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu

References VI

Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan, *Deepseek-v3 technical report*, 2025.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré, *Flashattention: Fast and memory-efficient exact attention with io-awareness*, 2022.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh, *Gptq: Accurate post-training quantization for generative pre-trained transformers*, 2023.

Georgi Gerganov, *ggml: Tensor library for machine learning*, https://github.com/ggerganov/ggml, 2023.

Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro, *Reducing activation recomputation in large transformer models*, May 2022, arXiv:2205.05198 [cs].

References VII

- Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joonsuk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee, *Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization*, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica, *Efficient memory management for large language model serving with pagedattention*, Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.
- Ilya Loshchilov and Frank Hutter, *Decoupled weight decay regularization*, 2019.
- Shenggui Li, Hongxin Liu, Zhengda Bian, Jiarui Fang, Haichen Huang, Yuliang Liu, Boxiang Wang, and Yang You, *Colossal-ai: A unified deep learning system for large-scale parallel training*, Proceedings of the 52nd International Conference on Parallel Processing (New York, NY, USA), ICPP '23, Association for Computing Machinery, 2023, p. 766–775.

References VIII

- Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen, *A survey on large language model acceleration based on kv cache management*, 2025.
- Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra, *Llm-qat: Data-free quantization aware training for large language models*, 2023.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han, *Awq: Activation-aware weight quantization for llm compression and acceleration*, 2024.
- Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He, *Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale*, 2022.

References IX

- Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He, *Zero-offload: Democratizing billion-scale model training*, 2021.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He, *Zero: Memory optimizations toward training trillion parameter models*, 2020.
- Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He, *Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters*, Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (New York, NY, USA), KDD '20, Association for Computing Machinery, 2020, p. 3505–3506.
- Noam Shazeer, *Fast transformer decoding: One write-head is all you need*, 2019.

References X

- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro, *Megatron-lm: Training multi-billion parameter language models using model parallelism*, 2020.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang, *Flexgen: High-throughput generative inference of large language models with a single gpu*, 2023.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han, *Smoothquant: Accurate and efficient post-training quantization for large language models*, 2024.

References XI

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu, *Qwen3 technical report*, 2025.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li, *Pytorch fsdp: Experiences on scaling fully sharded data parallel*, 2023.

References XII

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng, *Sglang: efficient execution of structured language model programs*, Proceedings of the 38th International Conference on Neural Information Processing Systems (Red Hook, NY, USA), NIPS '24, Curran Associates Inc., 2024.